UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Spin Table Tennis Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Spin Table Tennis

Spin Table Tennis is a standalone editable Unity game template. Move the paddle into the ball path; hits are automatic. Drag in the portrait layout or move the mouse in the desktop layout. Return rallies against the local computer opponent and meet the current match conditions.

Includes 25 curated levels, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Additional included libraries: 5 venues. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Spin Table Tennis > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/SpinTennis/Scenes/SpinTennis_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/SpinTennis`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/spintennis contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/SpinTennisGame.cs`
- `Scripts/SpinTennisGame.UI.cs`
- `Scripts/SpinTennisGame.View.cs`
- `Scripts/StData.cs`
- `Scripts/StGen.cs`
- `Scripts/StLevels.cs`
- `Scripts/StMeshes.cs`
- `Scripts/StSfx.cs`
- `Scripts/StSim.cs`

Main flow: SpinTennisGame.cs. UI and popup consumers: SpinTennisGame.UI.cs. Model/generator/loading: StData.cs, StGen.cs, StLevels.cs, StSim.cs. Authoring: Editor/SpinTennisLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Move the paddle into the ball path; hits are automatic. Drag in the portrait layout or move the mouse in the desktop layout. Return rallies against the local computer opponent and meet the current match conditions.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact helper names/unlocks/charges: Scripts/SpinTennisGame.cs; help and effect descriptions: Scripts/SpinTennisGame.cs.

Helper	Unlock	Free charges	Action and effect
Slow-Mo	3	3	Time slows down for 6 seconds. Perfect for reading tricky spin!
Wide Paddle	6	3	Your paddle grows wider for the next 2 points.
Power Smash	10	2	Your next return becomes a blazing smash to the far corner.
Second Chance	15	2	Lost a point? Tap it right away to cancel that point.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Spin Table Tennis > Level Editor**. Exact implementation: `Editor/SpinTennisLevelEditor.cs`.

Duplicate an early entry in this game-specific visual editor, change its visible name and one rule, save, validate, then play it. Use the exact source-control table below for this game.

1. Back up Resources/spintennis/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`SpinTennisLevelEditor.cs:45`	`[MenuItem("Szekipapa77/Spin Table Tennis/Level Editor")]`
`SpinTennisLevelEditor.cs:48`	`[MenuItem("Szekipapa77/Spin Table Tennis/Validate All Levels")]`
`SpinTennisLevelEditor.cs:120`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`SpinTennisLevelEditor.cs:121`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`SpinTennisLevelEditor.cs:124`	`if (GUILayout.Button("Validate", EditorStyles.toolbarButton, GUILayout.Width(70)) && Cur != null) ValidateOne(_sel);`
`SpinTennisLevelEditor.cs:125`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAll();`

SpinTennisLevelEditor.cs:130	if (GUILayout.Button(new GUIContent("Play this level", "Optional quick test: saves, opens the WebGL scene at this stop and enters Play mode. Editing never needs it."), EditorStyles.toolbarButton, GUILayout.Width(100)) && Cur != null) Play(_sel + 1);
SpinTennisLevelEditor.cs:158	if (GUILayout.Button("Add"))
SpinTennisLevelEditor.cs:163	if (GUILayout.Button("Dup") && Cur != null) { var c = Cur.Clone(); c.name += " copy"; Insert(_sel + 1, c); }
SpinTennisLevelEditor.cs:164	if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete", $"Delete stop {_sel + 1} '{Cur.name}'?", "Delete", "Cancel"))
SpinTennisLevelEditor.cs:168	if (GUILayout.Button("Up") && _sel > 0) Move(-1);
SpinTennisLevelEditor.cs:169	if (GUILayout.Button("Down") && _sel < _file.levels.Length - 1) Move(1);
SpinTennisLevelEditor.cs:191	EditorGUILayout.LabelField($"Stop {_sel + 1}", EditorStyles.boldLabel);
SpinTennisLevelEditor.cs:193	l.name = EditorGUILayout.TextField("Stop name", l.name);
SpinTennisLevelEditor.cs:194	l.venue = EditorGUILayout.Popup("Venue", Mathf.Clamp(l.venue, 0, _file.venues.Length - 1), _file.venues.Select(v => v.name).ToArray(), GUILayout.Width(300));
SpinTennisLevelEditor.cs:196	l.hint = EditorGUILayout.TextField("First-time hint", l.hint);
SpinTennisLevelEditor.cs:198	l.difficulty = EditorGUILayout.Slider("Design difficulty", l.difficulty, 0, 1);
SpinTennisLevelEditor.cs:199	l.breather = EditorGUILayout.ToggleLeft("Breather", l.breather, GUILayout.Width(80));
SpinTennisLevelEditor.cs:210	_showRules = EditorGUILayout.Foldout(_showRules, "Match rules", true);

SpinTennisLevelEditor.cs:213	`l.mode = EditorGUILayout.Popup("Mode", Mathf.Clamp(l.mode, 0, 3), ModeNames);`
SpinTennisLevelEditor.cs:217	`l.points = EditorGUILayout.IntSlider("Points to win", Mathf.Max(1, l.points), 1, 21);`
SpinTennisLevelEditor.cs:218	`l.winBy2 = EditorGUILayout.Toggle("Win by 2", l.winBy2);`
SpinTennisLevelEditor.cs:221	`l.goal = EditorGUILayout.IntSlider("Zone hits needed", Mathf.Max(1, l.goal), 1, 12);`
SpinTennisLevelEditor.cs:222	`l.points = EditorGUILayout.IntSlider("Lives (opponent points)", Mathf.Max(1, l.points), 1, 10);`
SpinTennisLevelEditor.cs:223	`l.zoneW = EditorGUILayout.Slider("Zone width (m)", l.zoneW, 0.2f, 1.5f);`
SpinTennisLevelEditor.cs:224	`l.zoneD = EditorGUILayout.Slider("Zone depth (m)", l.zoneD, 0.2f, 1.1f);`
SpinTennisLevelEditor.cs:227	`l.goal = EditorGUILayout.IntSlider("Aces needed", Mathf.Max(1, l.goal), 1, 10);`
SpinTennisLevelEditor.cs:228	`l.points = EditorGUILayout.IntSlider("Lives (opponent points)", Mathf.Max(1, l.points), 1, 10);`
SpinTennisLevelEditor.cs:231	`l.goal = EditorGUILayout.IntSlider("Returns in a row", Mathf.Max(1, l.goal), 2, 40);`
SpinTennisLevelEditor.cs:232	`l.points = EditorGUILayout.IntSlider("Misses allowed", Mathf.Max(1, l.points), 1, 10);`
SpinTennisLevelEditor.cs:235	`l.wind = EditorGUILayout.Slider("Wind (m/s², + right)", l.wind, -2f, 2f);`
SpinTennisLevelEditor.cs:236	`EditorGUILayout.LabelField("Goal text", l.GoalText(), EditorStyles.miniLabel);`
SpinTennisLevelEditor.cs:238	`_showOpp = EditorGUILayout.Foldout(_showOpp, "Opponent profile", true);`
SpinTennisLevelEditor.cs:243	`o.name = EditorGUILayout.TextField("Name", o.name);`
SpinTennisLevelEditor.cs:246	`o.style = EditorGUILayout.Popup("Style", Mathf.Clamp(o.style, 0, 4), StyleNames);`

SpinTennisLevelEditor.cs:249	o.reaction = EditorGUILayout.Slider(new GUIContent("Reaction delay", "0 = instant, 1 = very slow to start moving"), o.reaction, 0, 1);
SpinTennisLevelEditor.cs:250	o.speed = EditorGUILayout.Slider("Move speed", o.speed, 0, 1);
SpinTennisLevelEditor.cs:251	o.accuracy = EditorGUILayout.Slider(new GUIContent("Accuracy", "aim, reading spin and wind, fewer unforced errors"), o.accuracy, 0, 1);
SpinTennisLevelEditor.cs:252	o.spin = EditorGUILayout.Slider("Spin", o.spin, 0, 1);
SpinTennisLevelEditor.cs:253	o.power = EditorGUILayout.Slider("Power", o.power, 0, 1);
SpinTennisLevelEditor.cs:254	if (GUILayout.Button("Re-roll opponent from design difficulty"))
SpinTennisLevelEditor.cs:268	EditorGUILayout.HelpBox($"Validation ({res.Matches} AI-vs-AI matches, player model vs this opponent): {res}", res.Solvable ? MessageType.Info : MessageType.Warning);
SpinTennisLevelEditor.cs:344	EditorGUILayout.LabelField("orange: opponent landings - teal: player model landings", EditorStyles.miniLabel);
SpinTennisLevelEditor.cs:345	if (GUILayout.Button("Re-simulate shot spread")) { _spreadKey = ""; Repaint(); }
SpinTennisLevelEditor.cs:410	if (GUILayout.Button("Add")) { var l = _file.venues.ToList(); l.Add(_file.venues[Mathf.Clamp(_venueSel, 0, l.Count - 1)].Clone()); l[l.Count - 1].name += " copy"; _file.venues = l.ToArray(); _venueSel = l.Count - 1; _dirty = true; }
SpinTennisLevelEditor.cs:411	if (GUILayout.Button("Del") && _file.venues.Length > 1 && EditorUtility.DisplayDialog("Delete venue", "Stops using it fall back to the last venue. Delete?", "Delete", "Cancel"))
SpinTennisLevelEditor.cs:420	v.name = EditorGUILayout.TextField("Name", v.name);

SpinTennisLevelEditor.cs:421	v.look = EditorGUILayout.Popup("Set (props)", Mathf.Clamp(v.look, 0, 4), LookNames);
SpinTennisLevelEditor.cs:422	v.table = ToHex(EditorGUILayout.ColorField("Table", Hex(v.table)));
SpinTennisLevelEditor.cs:423	v.floor = ToHex(EditorGUILayout.ColorField("Floor", Hex(v.floor)));
SpinTennisLevelEditor.cs:424	v.wall = ToHex(EditorGUILayout.ColorField("Wall", Hex(v.wall)));
SpinTennisLevelEditor.cs:425	v.skyTop = ToHex(EditorGUILayout.ColorField("Sky top", Hex(v.skyTop)));
SpinTennisLevelEditor.cs:426	v.skyBottom = ToHex(EditorGUILayout.ColorField("Sky bottom", Hex(v.skyBottom)));
SpinTennisLevelEditor.cs:427	v.accent = ToHex(EditorGUILayout.ColorField("Accent", Hex(v.accent)));
SpinTennisLevelEditor.cs:428	v.wind = EditorGUILayout.Slider(new GUIContent("Default wind", "used by Auto Generate for stops at this venue"), v.wind, 0, 2);
SpinTennisLevelEditor.cs:429	v.bounce = EditorGUILayout.Slider(new GUIContent("Table bounce", "vertical restitution: higher = livelier table"), v.bounce, 0.7f, 0.95f);
SpinTennisLevelEditor.cs:448	EditorGUILayout.LabelField($"{used} stop(s) use this venue.", EditorStyles.miniLabel);
SpinTennisLevelEditor.cs:456	_showGen = EditorGUILayout.Foldout(_showGen, "Auto Generate (seeded, AI-vs-AI validated, same generator as the runtime)", true);
SpinTennisLevelEditor.cs:459	_genCount = EditorGUILayout.IntField("Count", _genCount); _genStart = EditorGUILayout.IntField("Start at stop #", _genStart);

`SpinTennisLevelEditor.cs:460`	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed);`
`SpinTennisLevelEditor.cs:463`	`_genFrom = EditorGUILayout.Slider("Start difficulty", _genFrom, 0, 1); _genTo = EditorGUILayout.Slider("End difficulty", _genTo, 0, 1);`
`SpinTennisLevelEditor.cs:466`	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 0, 8);`
`SpinTennisLevelEditor.cs:467`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`SpinTennisLevelEditor.cs:478`	`if (GUILayout.Button($"Generate {_genCount} stop(s)", GUILayout.Height(26))) Generate();`

CHAPTER 07

Generator, insertion and replacement

Use this editor generation panel, choose a reproducible seed and small count, review start/index and replace behavior, then generate, save and reopen. The source excerpt below identifies exact append/replace semantics.

Source	Exact current code
`SpinTennisLevelEditor.cs:43`	`bool _genReplace, _showGen = true, _showOpp = true, _showRules = true;`
`SpinTennisLevelEditor.cs:467`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`SpinTennisLevelEditor.cs:507`	`if (_genReplace) { int rem = Mathf.Min(made.Count, list.Count - at); if (rem > 0) list.RemoveRange(at, rem); }`
`SpinTennisLevelEditor.cs:508`	`list.InsertRange(at, made);`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/SpinTennis/Resources/spintennis/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "Mode": 0,
  "name": "Welcome Rally",
  "venue": 0,
  "mode": 0,
  "points": 3,
  "winBy2": false,
  "goal": 0,
  "zoneX": 0,
  "zoneZ": 0.9,
  "zoneW": 0.6,
  "zoneD": 0.45,
  "wind": 0,
  "difficulty": 0.02,
  "breather": false,
  "hint": "",
  "tut": 1,
  "opp": {
    "Style": 0,
    "name": "Pip Pebble",
    "color": "#7C6BE0",
    "style": 0,
    "reaction": 0.95782083,
    "speed": 0.11929827,
    "accuracy": 0.22544493,
    "spin": 0,
    "power": 0.10674773
  }
}
```


Root keys: venues, levels. Inspect StData.cs, StGen.cs, StLevels.cs, StSim.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/SpinTennisGame.UI.cs`. Find `Ink`, currently `#1D2B4F`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`SpinTennisGame.UI.cs:66 / void BuildHud()`	`var lp = HzUI.Panel("StopPill", _hud, Ink, 44);`
`SpinTennisGame.UI.cs:79 / void BuildHud()`	`var set = HzIcons.IconButton(_hud, HzIcons.Settings, Ink, () => { if (!_ended) ShowSettingsPopup(); }, 104);`
`SpinTennisGame.UI.cs:148 / void BuildHud()`	`_tipText = HzUI.Label("T", _tipPanel, "", portrait ? 40 : 38, Ink); _tipText.rectTransform.Stretch(26);`
`SpinTennisGame.UI.cs:220 / void BuildBoard(float bw, float bh)`	`else { var ic = HzIcons.Icon(_board, ModeIcon(m), 60, Ink); ic.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(80, 80), new Vector2(0, 18)); }`
`SpinTennisGame.UI.cs:221 / void BuildBoard(float bw, float bh)`	`_midText = HzUI.Label("Goal", _board, _def.GoalText(), 26, Ink, TextAnchor.MiddleCenter, false); _midText.fontStyle = FontStyle.Bold;`
`SpinTennisGame.UI.cs:259 / void BuildBoard(float bw, float bh)`	`v.Caption = HzUI.Label("Cap", v.Root, "", 26, Ink);`
`SpinTennisGame.UI.cs:281 / void RefreshBoosters()`	`v.Caption.color = un ? Ink : HzUI.Hex("#6B7894");`
`SpinTennisGame.UI.cs:544 / HzPopup CourtPopup(string title, string message, Color accent, Sprite art, string icon, params (string label, Color color, Action action)[] buttons)`	`p.Message = HzUI.Label("Msg", card.transform, message, 40, Ink, TextAnchor.MiddleCenter, false);`
`SpinTennisGame.UI.cs:614 / void VersusPopup(Action done)`	`var nm = HzUI.Label("Name", panel.transform, o.name, 48, Ink, TextAnchor.MiddleLeft);`

SpinTennisGame.UI.cs:623 / void VersusPopup(Action done)	var lab = HzUI.Label("L", panel.transform, names[i], 24, Ink, TextAnchor.MiddleLeft, false); lab.fontStyle = FontStyle.Bold;
SpinTennisGame.UI.cs:647 / void ShowSettingsPopup()	var p = CourtPopup("Settings", "", Ink, null, null,

Source	Exact current code
SpinTennisGame.cs:30 / class/static declarations	static readonly string[] BoostColor = { "#4FA8FF", "#1FB5A8", "#FF6B57", "#9B6BFF" };
SpinTennisGame.cs:149 / IEnumerator Intro(string key, string title, string msg, string art, string icon = null)	CourtPopup(title, msg, HzUI.Hex("#1FB5A8"), Art(art), icon, ("Got it!", HzUI.Hex("#FF6B57"), () => closed = true));
SpinTennisGame.cs:275 / void ProcessEvents()	SpawnSparks(pos, me ? HzUI.Hex("#FFE7A3") : OppColor, hard ? 14 : 7, hard ? 1.4f : 0.9f);
SpinTennisGame.cs:277 / void ProcessEvents()	if (e.Kind == Ev.Smash) { HzAudio.Play(Sfx.Whoosh, 0.7f); ShakeCamera(0.12f, 0.3f); FloatText(pos + Vector3.up * 0.3f, "SMASH!", HzUI.Hex("#FF6B57")); }
SpinTennisGame.cs:285 / void ProcessEvents()	SpawnRing(new Vector3(e.X, 0.004f, e.Z), e.Side == 0 ? HzUI.Hex("#FFC845") : Color.white, 0.32f);
SpinTennisGame.cs:298 / void ProcessEvents()	FloatText(new Vector3(Mathf.Clamp(e.X, -1.2f, 1.2f), 0.4f, Mathf.Clamp(e.Z, -1.8f, 1.8f)), "OUT", HzUI.Hex("#FFD3CC"));
SpinTennisGame.cs:301 / void ProcessEvents()	FloatText(pos + Vector3.up * 0.35f, "FAULT", HzUI.Hex("#FFD3CC"));
SpinTennisGame.cs:308 / void ProcessEvents()	FloatText(pos + Vector3.up * 0.3f, "PERFECT!", HzUI.Hex("#FFC845"));
SpinTennisGame.cs:309 / void ProcessEvents()	SpawnSparks(pos, HzUI.Hex("#FFC845"), 12, 1.2f);
SpinTennisGame.cs:313 / void ProcessEvents()	SpawnSparks(pos + Vector3.up * 0.05f, HzUI.Hex("#FFC845"), 18, 1.3f);
SpinTennisGame.cs:314 / void ProcessEvents()	SpawnRing(new Vector3(e.X, 0.006f, e.Z), HzUI.Hex("#FFC845"), 0.9f);

`SpinTennisGame.cs:315 / void ProcessEvents()`	`FloatText(pos + Vector3.up * 0.35f, "ZONE " + (int)e.Value + "/" + _def.goal, HzUI.Hex("#FFC845"));`
`SpinTennisGame.cs:321 / void ProcessEvents()`	`FloatText(new Vector3(0, 0.6f, 1.2f), "ACE!", HzUI.Hex("#FFC845"));`
`SpinTennisGame.cs:381 / IEnumerator WinFlow()`	`var pop = CourtPopup("Victory!", msg, HzUI.Hex("#FF6B57"), Art("win_badge"), null,`
`SpinTennisGame.cs:382 / IEnumerator WinFlow()`	`("Next Match", HzUI.Hex("#1FB5A8"), () => HzPlatform.Interstitial(GameId, () => LoadLevel(Level))));`
`SpinTennisGame.cs:422 / IEnumerator LoseFlow()`	`CourtPopup(title, msg + "\nCancel the last point and play on?", HzUI.Hex("#7C6BE0"), Art("lose_badge"), null,`
`SpinTennisGame.cs:423 / IEnumerator LoseFlow()`	`("Second Chance (ad)", HzUI.Hex("#1FB5A8"), () => HzPlatform.Rewarded(GameId, "rescue", ok => { if (ok) Rescue(); else LoadLevel(Level, true); })),`
`SpinTennisGame.cs:424 / IEnumerator LoseFlow()`	`("Retry", HzUI.Hex("#FF9F5A"), () => LoadLevel(Level, true)));`
`SpinTennisGame.cs:427 / IEnumerator LoseFlow()`	`CourtPopup(title, msg, HzUI.Hex("#7C6BE0"), Art("lose_badge"), null, ("Retry", HzUI.Hex("#FF9F5A"), () => LoadLevel(Level, true)));`
`SpinTennisGame.cs:453 / void OnBooster(int b)`	`("Get 1 (ad)", HzUI.Hex("#1FB5A8"), () => HzPlatform.Rewarded(GameId, "booster_" + BoostShort[b].ToLowerInvariant(), ok => { if (ok) { SetCharges(b, Charges(b) + 1); Toast("+1 " + BoostName[b]); HzAudio.Play(Sfx.Coin, 0.7f); } })),`
`SpinTennisGame.cs:454 / void OnBooster(int b)`	`("No thanks", HzUI.Hex("#B8ABD1"), null));`
`SpinTennisGame.cs:464 / void OnBooster(int b)`	`case BSmash: _sim.SmashArmed = true; HzAudio.Play(Sfx.Smash, 0.85f); SpawnSparks(PaddlePos(0), HzUI.Hex("#FF6B57"), 16, 1.2f); break;`
`SpinTennisGame.cs:465 / void OnBooster(int b)`	`case BLet: _sim.UndoLastPoint(); HzAudio.Play(Sfx.Let, 0.85f); FloatText(new Vector3(0, 0.5f, 0.2f), "POINT CANCELLED", HzUI.Hex("#C9B6FF")); UpdateScoreUi(true); break;`

SpinTennisGame.UI.cs:12 / class/static declarations	static readonly Color Ink = HzUI.Hex("#1D2B4F");
SpinTennisGame.UI.cs:13 / class/static declarations	static readonly Color Teal = HzUI.Hex("#1FB5A8");
SpinTennisGame.UI.cs:14 / class/static declarations	static readonly Color Coral = HzUI.Hex("#FF6B57");
SpinTennisGame.UI.cs:15 / class/static declarations	static readonly Color Sun = HzUI.Hex("#FFC845");
SpinTennisGame.UI.cs:16 / class/static declarations	static readonly Color Cream = HzUI.Hex("#FFF6E5");
SpinTennisGame.UI.cs:98 / void BuildHud()	var ic = HzIcons.Icon(wc.transform, _def.wind > 0 ? HzIcons.ArrowForward : HzIcons.ArrowBack, 44, HzUI.Hex("#BFE9FF"));
SpinTennisGame.UI.cs:273 / void RefreshBoosters()	v.Inner.color = un ? HzUI.Hex(BoostColor[b]) : HzUI.Hex("#B9C3D6");
SpinTennisGame.UI.cs:281 / void RefreshBoosters()	v.Caption.color = un ? Ink : HzUI.Hex("#6B7894");
SpinTennisGame.UI.cs:496 / HzPopup CourtPopup(string title, string message, Color accent, Sprite art, string icon, params (string label, Color color, Action action)[] buttons)	var inner = HzUI.Panel("Inner", card.transform, HzUI.Hex("#FFFDF7"), 44); inner.raycastTarget = false;
SpinTennisGame.UI.cs:576 / void AddStars(HzPopup pop, int stars)	st.AddOutline(HzUI.Hex("#B5761E"), on ? 4 : 0.01f);
SpinTennisGame.UI.cs:616 / void VersusPopup(Action done)	var vn = HzUI.Label("Venue", panel.transform, _venue.name + (Mathf.Abs(_def.wind) > 0.05f ? " - windy" : ""), 28, HzUI.Hex("#5A6A8A"), TextAnchor.MiddleLeft, false); vn.fontStyle = FontStyle.Bold;
SpinTennisGame.UI.cs:648 / void ShowSettingsPopup()	(SoundLabel(), HzUI.Hex("#4FA8FF"), null),
SpinTennisGame.UI.cs:649 / void ShowSettingsPopup()	(MusicLabel(), HzUI.Hex("#9B6BFF"), null),
SpinTennisGame.UI.cs:650 / void ShowSettingsPopup()	("Restart Match", HzUI.Hex("#FF9F5A"), () => LoadLevel(Level, true)),

SpinTennisGame.View.cs:45 / class/static declarations	Color OppColor => HzUI.Hex(string.IsNullOrEmpty(_def.opp.color) ? "#FF6B57" : _def.opp.color);
SpinTennisGame.View.cs:301 / void BuildWorld()	_sun.transform.rotation = Quaternion.Euler(62, -20, 0); _sun.color = HzUI.Hex("#C9D2FF"); _sun.intensity = 0.75f;
SpinTennisGame.View.cs:302 / void BuildWorld()	_fill.color = HzUI.Hex("#FF6FD8"); _fill.intensity = 0.25f; _rim.color = HzUI.Hex("#4FE3FF"); _rim.intensity = 0.35f;
SpinTennisGame.View.cs:303 / void BuildWorld()	RenderSettings.ambientSkyColor = HzUI.Hex("#3B3570"); RenderSettings.ambientEquatorColor = HzUI.Hex("#2A2850"); RenderSettings.ambientGroundColor = HzUI.Hex("#141428");
SpinTennisGame.View.cs:306 / void BuildWorld()	_sun.transform.rotation = Quaternion.Euler(55, -35, 0); _sun.color = HzUI.Hex("#FFE2BF"); _sun.intensity = 1.0f;
SpinTennisGame.View.cs:307 / void BuildWorld()	_fill.color = HzUI.Hex("#BFD6FF"); _fill.intensity = 0.3f; _rim.color = HzUI.Hex("#FFB070"); _rim.intensity = 0.35f;
SpinTennisGame.View.cs:308 / void BuildWorld()	RenderSettings.ambientSkyColor = HzUI.Hex("#E9DCCB"); RenderSettings.ambientEquatorColor = HzUI.Hex("#B89C80"); RenderSettings.ambientGroundColor = HzUI.Hex("#5A4232");
SpinTennisGame.View.cs:311 / void BuildWorld()	_sun.transform.rotation = Quaternion.Euler(68, -15, 0); _sun.color = HzUI.Hex("#FFF4E0"); _sun.intensity = 1.15f;
SpinTennisGame.View.cs:312 / void BuildWorld()	_fill.color = HzUI.Hex("#8FB0FF"); _fill.intensity = 0.3f; _rim.color = HzUI.Hex("#FFD27A"); _rim.intensity = 0.5f;
SpinTennisGame.View.cs:313 / void BuildWorld()	RenderSettings.ambientSkyColor = HzUI.Hex("#4A5378"); RenderSettings.ambientEquatorColor = HzUI.Hex("#353C58"); RenderSettings.ambientGroundColor = HzUI.Hex("#1A1E2E");

SpinTennisGame.View.cs:316 / void BuildWorld()	_sun.transform.rotation = Quaternion.Euler(58, -40, 0); _sun.color = HzUI.Hex("#FFF1D6"); _sun.intensity = 1.25f;
SpinTennisGame.View.cs:317 / void BuildWorld()	_fill.color = HzUI.Hex("#9FD8FF"); _fill.intensity = 0.35f; _rim.color = HzUI.Hex("#FFD3A8"); _rim.intensity = 0.35f;
SpinTennisGame.View.cs:318 / void BuildWorld()	RenderSettings.ambientSkyColor = HzUI.Hex("#BFE6FF"); RenderSettings.ambientEquatorColor = HzUI.Hex("#F5E1BE"); RenderSettings.ambientGroundColor = HzUI.Hex("#B89A6A");
SpinTennisGame.View.cs:321 / void BuildWorld()	_sun.transform.rotation = Quaternion.Euler(52, -32, 0); _sun.color = HzUI.Hex("#FFEFD8"); _sun.intensity = 1.15f;
SpinTennisGame.View.cs:322 / void BuildWorld()	_fill.color = HzUI.Hex("#BFE0FF"); _fill.intensity = 0.32f; _rim.color = HzUI.Hex("#FFE0B8"); _rim.intensity = 0.35f;
SpinTennisGame.View.cs:323 / void BuildWorld()	RenderSettings.ambientSkyColor = HzUI.Hex("#CDEBFF"); RenderSettings.ambientEquatorColor = HzUI.Hex("#DCEBD0"); RenderSettings.ambientGroundColor = HzUI.Hex("#6E8F5A");
SpinTennisGame.View.cs:342 / void BuildTable()	var line = Lit("#FBFBF6", 0.35f);
SpinTennisGame.View.cs:343 / void BuildTable()	var metal = Lit("#2B2F3A", 0.55f, 0.6f);
SpinTennisGame.View.cs:372 / void BuildBall()	var ballMat = Emit(HzUI.Hex("#FFF4E0"), HzUI.Hex("#FFB070") * 0.35f, 0.8f);
SpinTennisGame.View.cs:386 / void BuildBall()	_landingR = Part("Landing", _world, StMeshes.Ring(0.62f), Glow(HzUI.Hex("#FFC845") * 0.9f), new Vector3(0, 0.003f, -0.8f), new Vector3(0.16f, 1, 0.16f), default, false);
SpinTennisGame.View.cs:399 / void BuildZone()	var edge = Glow(HzUI.Hex("#FFC845") * 1.4f);
SpinTennisGame.View.cs:444 / void BuildPaddles()	var wood = Lit("#E8C38E", 0.35f);
SpinTennisGame.View.cs:445 / void BuildPaddles()	var handle = Lit("#B9804A", 0.45f);

SpinTennisGame.View.cs:459 / void BuildPaddles()	Part("Rim", blade, StMeshes.Ring(0.8f), Lit("#FFF6E5", 0.5f), new Vector3(0, 0, -0.01f), new Vector3(0.27f, 1f, 0.28f), new Vector3(-90, 0, 0), true);
SpinTennisGame.View.cs:460 / void BuildPaddles()	Part("RimBack", blade, StMeshes.Ring(0.8f), Lit("#1FB5A8", 0.5f), new Vector3(0, 0, -0.008f), new Vector3(0.27f, 1f, 0.28f), new Vector3(90, 0, 0), true);
SpinTennisGame.View.cs:488 / void BuildOpponent()	var shorts = Lit("#1D2B4F", 0.3f);
SpinTennisGame.View.cs:489 / void BuildOpponent()	string[] skins = { "#F2C7A5", "#D9A27C", "#A86F4C", "#7A4E33", "#F6D7BD" };
SpinTennisGame.View.cs:492 / void BuildOpponent()	var band = Lit(h % 2 == 0 ? "#FFFFFF" : "#FFC845", 0.4f);
SpinTennisGame.View.cs:493 / void BuildOpponent()	var eye = Lit("#1D2B4F", 0.8f);
SpinTennisGame.View.cs:494 / void BuildOpponent()	var shoe = Lit("#FFFFFF", 0.4f);
SpinTennisGame.View.cs:505 / void BuildOpponent()	Part("Hair", _oppHead, StMeshes.Sphere, Lit(h % 3 == 0 ? "#3B2A20" : h % 3 == 1 ? "#E0B050" : "#5A3A2A", 0.3f), new Vector3(0, 0.08f, 0.03f), new Vector3(0.29f, 0.2f, 0.28f));
SpinTennisGame.View.cs:526 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);	Part("Mat", root, StMeshes.RBox(5.2f, 0.02f, 9f, 0.02f, 1), Lit("#C9E7B8", 0.2f), new Vector3(0, Sim.FloorY + 0.01f, 0), Vector3.one, default, false);
SpinTennisGame.View.cs:527 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);	var hedge = Lit("#3E9B57", 0.25f); var hedge2 = Lit("#58B26A", 0.25f);
SpinTennisGame.View.cs:529 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);	var trunk = Lit("#8A5A3B", 0.3f); var leaf = Lit("#4FAE5E", 0.3f); var leaf2 = Lit("#7CC36E", 0.3f);
SpinTennisGame.View.cs:537 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);	var bench = Lit("#FFF6E5", 0.3f);
SpinTennisGame.View.cs:539 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);	var flowerA = Lit("#FF8FB8", 0.4f); var flowerB = Lit("#FFE07A", 0.4f);

`SpinTennisGame.View.cs:545 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Deck", root, StMeshes.RBox(5.4f, 0.06f, 9.2f, 0.02f, 1), Lit("#C99A6B", 0.3f), new Vector3(0, Sim.FloorY + 0.03f, 0), Vector3.one);`
`SpinTennisGame.View.cs:546 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var plank = Lit("#B88859", 0.3f);`
`SpinTennisGame.View.cs:548 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Sea", root, StMeshes.RBox(80, 0.05f, 30, 0.02f, 1), Lit("#2EB5D8", 0.85f), new Vector3(0, Sim.FloorY + 0.02f, 24), Vector3.one, default, false);`
`SpinTennisGame.View.cs:549 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Foam", root, StMeshes.RBox(80, 0.06f, 0.4f, 0.02f, 1), Lit("#F4FBFF", 0.5f), new Vector3(0, Sim.FloorY + 0.03f, 9.2f), Vector3.one, default, false);`
`SpinTennisGame.View.cs:550 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var trunk = Lit("#A57A50", 0.3f); var frond = Lit("#3FA45B", 0.35f);`
`SpinTennisGame.View.cs:558 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var poleM = Lit("#FFF6E5", 0.4f);`
`SpinTennisGame.View.cs:559 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`string[] umb = { "#FF6B57", "#FFC845", "#1FB5A8" };`
`SpinTennisGame.View.cs:571 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var gridMat = Glow(HzUI.Hex("#7A5CFF") * 0.6f);`
`SpinTennisGame.View.cs:574 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var wall = Lit("#22264A", 0.4f);`
`SpinTennisGame.View.cs:578 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var pink = Glow(HzUI.Hex("#FF4FD8") * 1.6f); var cyan = Glow(HzUI.Hex("#4FE3FF") * 1.6f); var gold = Glow(HzUI.Hex("#FFC845") * 1.4f);`
`SpinTennisGame.View.cs:583 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Pillar", root, StMeshes.RBox(0.5f, 6, 0.5f, 0.08f, 2), Lit("#2F3466", 0.5f), new Vector3(i * 3.2f, Sim.FloorY + 3, 8.0f), Vector3.one, default, false);`

`SpinTennisGame.View.cs:595 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var plank = Lit("#A9784A", 0.35f);`
`SpinTennisGame.View.cs:597 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Rug", root, StMeshes.RBox(4.6f, 0.02f, 7.5f, 0.05f, 1), Lit("#C2414B", 0.2f), new Vector3(0, Sim.FloorY + 0.01f, 0), Vector3.one, default, false);`
`SpinTennisGame.View.cs:598 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("RugBorder", root, StMeshes.RBox(4.9f, 0.015f, 7.8f, 0.05f, 1), Lit("#F2E6D0", 0.2f), new Vector3(0, Sim.FloorY + 0.006f, 0), Vector3.one, default, false);`
`SpinTennisGame.View.cs:599 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var wall = Lit("#EADBC8", 0.25f); var log = Lit("#8C5A36", 0.35f);`
`SpinTennisGame.View.cs:604 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var window = Emit(HzUI.Hex("#DDEEFF"), HzUI.Hex("#BFDFFF") * 1.1f, 0.2f);`
`SpinTennisGame.View.cs:605 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var frame = Lit("#6B4426", 0.35f);`
`SpinTennisGame.View.cs:611 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Snow", root, StMeshes.RBox(1.9f, 0.12f, 0.3f, 0.05f, 2), Lit("#FFFFFF", 0.3f), new Vector3(wx, Sim.FloorY + 1.3f, 6.2f), Vector3.one, default, false);`
`SpinTennisGame.View.cs:613 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Hearth", root, StMeshes.RBox(1.4f, 1.2f, 0.5f, 0.05f, 2), Lit("#7E7A78", 0.2f), new Vector3(-4.2f, Sim.FloorY + 0.6f, 6.0f), Vector3.one, default, false);`
`SpinTennisGame.View.cs:614 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Fire", root, StMeshes.RBox(0.8f, 0.5f, 0.1f, 0.05f, 2), Glow(HzUI.Hex("#FF8A3C") * 2.2f), new Vector3(-4.2f, Sim.FloorY + 0.45f, 5.74f), Vector3.one, default, false);`
`SpinTennisGame.View.cs:620 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Court", root, StMeshes.RBox(6f, 0.02f, 10f, 0.02f, 1), Lit("#C2413B", 0.35f), new Vector3(0, Sim.FloorY + 0.01f, 0), Vector3.one, default, false);`
`SpinTennisGame.View.cs:621 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("CourtLine", root, StMeshes.RBox(6.2f, 0.015f, 10.2f, 0.02f, 1), Lit("#FFFFFF", 0.35f), new Vector3(0, Sim.FloorY + 0.006f, 0), Vector3.one, default, false);`

`SpinTennisGame.View.cs:622 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var tier = Lit("#3A4466", 0.4f);`
`SpinTennisGame.View.cs:623 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`string[] crowd = { "#FF6B57", "#FFC845", "#1FB5A8", "#F4F4F4", "#7C6BE0" };`
`SpinTennisGame.View.cs:640 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("Roof", root, StMeshes.RBox(30, 0.4f, 30, 0.05f, 1), Lit("#1A2140", 0.3f), new Vector3(0, Sim.FloorY + 7.5f, 6), Vector3.one, default, false);`
`SpinTennisGame.View.cs:644 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`var banner = Lit("#FFC845", 0.4f);`
`SpinTennisGame.View.cs:646 / float Rf(float a, float b) => a + (float)rng.NextDouble() * (b - a);`	`Part("BannerBall", root, StMeshes.Sphere, Glow(HzUI.Hex("#FFFFFF") * 1.3f), new Vector3(0, Sim.FloorY + 4.6f, 9.42f), Vector3.one * 0.45f, default, false);`
`StData.cs:16 / class/static declarations`	`public string color = "#FF6B57";`
`StData.cs:38 / class/static declarations`	`public string table = "#1FB5A8";`
`StData.cs:39 / class/static declarations`	`public string floor = "#7CC36E";`
`StData.cs:40 / class/static declarations`	`public string wall = "#CFEFD9";`
`StData.cs:41 / class/static declarations`	`public string skyTop = "#8ED6FF";`
`StData.cs:42 / class/static declarations`	`public string skyBottom = "#FFF1C9";`
`StData.cs:43 / class/static declarations`	`public string accent = "#FFC845";`
`StGen.cs:44 / public override string ToString() => $"win {WinRate * 100:0}% rally {AvgRally:0.0} points {AvgPoints:0.0} {AvgSeconds:0}s diff {Difficulty:0.00}{(Solvable ? "" : " NOT OK")}";`	`public static readonly string[] Colors = { "#FF6B57", "#7C6BE0", "#2E9CDB", "#F2994A", "#27AE60", "#EB5A9B", "#16A3A3", "#C0392B", "#9B51E0", "#F2C94C" };`
`StGen.cs:49 / public override string ToString() => $"win {WinRate * 100:0}% rally {AvgRally:0.0} points {AvgPoints:0.0} {AvgSeconds:0}s diff {Difficulty:0.00}{(Solvable ? "" : " NOT OK")}";`	`new Venue { name = "Garden Club", table = "#1FB5A8", floor = "#7CC36E", wall = "#CFEFD9", skyTop = "#8ED6FF", skyBottom = "#FFF1C9", accent = "#FFC845", wind = 0, bounce = 0.86f, look = 0 },`

`StGen.cs:50 / public override string ToString() => $"win {WinRate * 100:0}% rally {AvgRally:0.0} points {AvgPoints:0.0} {AvgSeconds:0}s diff {Difficulty:0.00}{(Solvable ? "" : " NOT OK")}";`	`new Venue { name = "Beach Deck", table = "#FF6B57", floor = "#F4D7A1", wall = "#FFE9C7", skyTop = "#5EC8F2", skyBottom = "#FFE6B8", accent = "#1FB5A8", wind = 0.9f, bounce = 0.84f, look = 1 },`
`StGen.cs:51 / public override string ToString() => $"win {WinRate * 100:0}% rally {AvgRally:0.0} points {AvgPoints:0.0} {AvgSeconds:0}s diff {Difficulty:0.00}{(Solvable ? "" : " NOT OK")}";`	`new Venue { name = "Neon Hall", table = "#5B3FA8", floor = "#1C1F3A", wall = "#2A2F5C", skyTop = "#151733", skyBottom = "#3B2A66", accent = "#FF4FD8", wind = 0, bounce = 0.9f, look = 2 },`
`StGen.cs:52 / public override string ToString() => $"win {WinRate * 100:0}% rally {AvgRally:0.0} points {AvgPoints:0.0} {AvgSeconds:0}s diff {Difficulty:0.00}{(Solvable ? "" : " NOT OK")}";`	`new Venue { name = "Snow Lodge", table = "#2F7D4E", floor = "#B98552", wall = "#EADBC8", skyTop = "#B9D9F2", skyBottom = "#F4F7FB", accent = "#E8505B", wind = 0, bounce = 0.82f, look = 3 },`
`StGen.cs:53 / public override string ToString() => $"win {WinRate * 100:0}% rally {AvgRally:0.0} points {AvgPoints:0.0} {AvgSeconds:0}s diff {Difficulty:0.00}{(Solvable ? "" : " NOT OK")}";`	`new Venue { name = "Grand Dome", table = "#1D3F8F", floor = "#2B3247", wall = "#39425E", skyTop = "#1A2140", skyBottom = "#45507A", accent = "#FFC845", wind = 0, bounce = 0.88f, look = 4 },`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/SpinTennisGame.UI.cs`, find the displayed literal `"Settings"`, and replace its displayed value with `"Options"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact helper names/unlocks/charges: Scripts/SpinTennisGame.cs; help and effect descriptions: Scripts/SpinTennisGame.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`SpinTennisGame.cs:26 / class/static declarations`	`static readonly string[] BoostName = { "Slow-Mo", "Wide Paddle", "Power Smash", "Second Chance" };`
`SpinTennisGame.cs:149 / IEnumerator Intro(string key, string title, string msg, string art, string icon = null)`	`CourtPopup(title, msg, HzUI.Hex("#1FB5A8"), Art(art), icon, ("Got it!", HzUI.Hex("#FF6B57"), () => closed = true));`
`SpinTennisGame.cs:381 / IEnumerator WinFlow()`	`var pop = CourtPopup("Victory!", msg, HzUI.Hex("#FF6B57"), Art("win_badge"), null,`
`SpinTennisGame.cs:422 / IEnumerator LoseFlow()`	`CourtPopup(title, msg + "\nCancel the last point and play on?", HzUI.Hex("#7C6BE0"), Art("lose_badge"), null,`
`SpinTennisGame.cs:427 / IEnumerator LoseFlow()`	`CourtPopup(title, msg, HzUI.Hex("#7C6BE0"), Art("lose_badge"), null, ("Retry", HzUI.Hex("#FF9F5A"), () => LoadLevel(Level, true)));`
`SpinTennisGame.cs:448 / void OnBooster(int b)`	`if (!Unlocked(b)) { Toast("Unlocks at stop " + BoostUnlock[b]); HzAudio.Play(Sfx.Wrong, 0.5f); if (_boost[b] != null) HzTween.Shake(_boost[b].Root, 10, 0.3f); return; }`

SpinTennisGame.cs:452 / void OnBooster(int b)	CourtPopup("Need " + BoostName[b] + "?", "Watch a short ad to get one more.", HzUI.Hex(BoostColor[b]), Art(BoostArt[b]), BoostIcon[b],
SpinTennisGame.cs:453 / void OnBooster(int b)	("Get 1 (ad)", HzUI.Hex("#1FB5A8"), () => HzPlatform.Rewarded(GameId, "booster_" + BoostShort[b].ToLowerInvariant(), ok => { if (ok) { SetCharges(b, Charges(b) + 1); Toast("+1 " + BoostName[b]); HzAudio.Play(Sfx.Coin, 0.7f); } })),
SpinTennisGame.cs:457 / void OnBooster(int b)	if (Active(b)) { HzAudio.Play(Sfx.Wrong, 0.4f); Toast(BoostName[b] + " is already on"); return; }
SpinTennisGame.cs:458 / void OnBooster(int b)	if (b == BLet && !_sim.CanUndo) { HzAudio.Play(Sfx.Wrong, 0.4f); Toast("Use it right after you lose a point"); return; }
SpinTennisGame.cs:511 / public override string DebugCommand(string cmd)	case "settings": ShowSettingsPopup(); return "ok";
SpinTennisGame.UI.cs:72 / void BuildHud()	_stopText = HzUI.Label("Stop", lp.transform, "Stop " + Level, 52, Color.white);
SpinTennisGame.UI.cs:74 / void BuildHud()	_stopSub = HzUI.Label("Venue", lp.transform, _venue.name + (_def.breather ? " (easy)" : ""), 30, Sun, TextAnchor.MiddleCenter, false);
SpinTennisGame.UI.cs:101 / void BuildHud()	var wt = HzUI.Label("T", wc.transform, "Breeze " + new string('|', strength), 32, Color.white, TextAnchor.MiddleLeft, false); wt.fontStyle = FontStyle.Bold;
SpinTennisGame.UI.cs:107 / void BuildHud()	_rallyText = HzUI.Label("T", rc.transform, "", 34, Sun); _rallyText.rectTransform.Stretch(4);
SpinTennisGame.UI.cs:135 / void BuildHud()	var hint = HzUI.Label("T", hp.transform, "Move the mouse to play - swing forward for power, sideways to curve - keys 1-4: helpers", 28, Color.white, TextAnchor.MiddleCenter, false);
SpinTennisGame.UI.cs:148 / void BuildHud()	_tipText = HzUI.Label("T", _tipPanel, "", portrait ? 40 : 38, Ink); _tipText.rectTransform.Stretch(26);

SpinTennisGame.UI.cs:174 / void BuildBoard(float bw, float bh)	var lname = HzUI.Label("Name", left.transform, m == Mode.Match ? "YOU" : m == Mode.Targets ? "ZONE" : m == Mode.Serve ? "ACES" : "STREAK", 30, Cream);
SpinTennisGame.UI.cs:176 / void BuildBoard(float bw, float bh)	_leftSmall = HzUI.Label("Small", left.transform, "", 26, new Color(1, 1, 1, 0.8f), TextAnchor.MiddleCenter, false); _leftSmall.fontStyle = FontStyle.Bold;
SpinTennisGame.UI.cs:178 / void BuildBoard(float bw, float bh)	_leftBig = HzUI.Label("Big", left.transform, "0", 84, Color.white);
SpinTennisGame.UI.cs:187 / void BuildBoard(float bw, float bh)	_rightBig = HzUI.Label("Big", right.transform, "0", 84, Color.white);
SpinTennisGame.UI.cs:190 / void BuildBoard(float bw, float bh)	var rname = HzUI.Label("Name", right.transform, m == Mode.Match ? FirstName.ToUpperInvariant() : "LIVES", 30, Cream);
SpinTennisGame.UI.cs:193 / void BuildBoard(float bw, float bh)	_rightSmall = HzUI.Label("Small", right.transform, "", 26, new Color(1, 1, 1, 0.85f), TextAnchor.MiddleCenter, false); _rightSmall.fontStyle = FontStyle.Bold;
SpinTennisGame.UI.cs:221 / void BuildBoard(float bw, float bh)	_midText = HzUI.Label("Goal", _board, _def.GoalText(), 26, Ink, TextAnchor.MiddleCenter, false); _midText.fontStyle = FontStyle.Bold;
SpinTennisGame.UI.cs:231 / void BuildBoard(float bw, float bh)	v.Btn = HzUI.Button("Booster" + b, parent, null, Color.white, Color.white, () => OnBooster(id), 40, (int)(size / 2));
SpinTennisGame.UI.cs:257 / void BuildBoard(float bw, float bh)	v.BadgeText = HzUI.Label("N", bd.transform, "", 32, Color.white); v.BadgeText.rectTransform.Stretch();
SpinTennisGame.UI.cs:259 / void BuildBoard(float bw, float bh)	v.Caption = HzUI.Label("Cap", v.Root, "", 26, Ink);
SpinTennisGame.UI.cs:278 / void RefreshBoosters()	v.BadgeText.text = c > 0 ? c.ToString() : "+";
SpinTennisGame.UI.cs:280 / void RefreshBoosters()	v.Caption.text = un ? BoostShort[b] : "Stop " + BoostUnlock[b];
SpinTennisGame.UI.cs:298 / void UpdateHud(float dt)	if (on) { _rallyText.text = "Rally " + rally; HzTween.Punch(_rallyChip, 0.12f, 0.2f); }

`SpinTennisGame.UI.cs:320 / void UpdateScoreUi(bool punch)`	`case Mode.Targets: _leftBig.text = _sim.TargetHits.ToString(); _leftSmall.text = "of " + _def.goal; break;`
`SpinTennisGame.UI.cs:321 / void UpdateScoreUi(bool punch)`	`case Mode.Serve: _leftBig.text = _sim.Aces.ToString(); _leftSmall.text = "of " + _def.goal; break;`
`SpinTennisGame.UI.cs:322 / void UpdateScoreUi(bool punch)`	`case Mode.Rally: _leftBig.text = _sim.Streak.ToString(); _leftSmall.text = "of " + _def.goal; break;`
`SpinTennisGame.UI.cs:323 / void UpdateScoreUi(bool punch)`	`default: _leftBig.text = _sim.Score[0].ToString(); _leftSmall.text = ""; break;`
`SpinTennisGame.UI.cs:325 / void UpdateScoreUi(bool punch)`	`if (m == Mode.Match) { _rightBig.text = _sim.Score[1].ToString(); _rightSmall.text = ""; }`
`SpinTennisGame.UI.cs:331 / void UpdateScoreUi(bool punch)`	`_rightSmall.text = "";`
`SpinTennisGame.UI.cs:346 / void PointBanner(bool mine)`	`var l = HzUI.Label("T", bg.transform, txt, 52, Color.white); l.rectTransform.Stretch(8);`
`SpinTennisGame.UI.cs:358 / void FloatText(Vector3 world, string text, Color c)`	`var l = HzUI.Label("Float", _labels, text, 54, Color.white);`
`SpinTennisGame.UI.cs:433 / void RefreshGuide()`	`string action = _guide == BLet ? "You lost a point: tap Second Chance to cancel it. It's free!" : $"New: {BoostName[_guide]}! Tap it to try it. This one is free!";`
`SpinTennisGame.UI.cs:447 / void CompleteGuide(int b)`	`Toast("Nice! " + BoostName[b] + " is yours", Cream, 1.3f, IsPortrait ? 0.62f : 0.5f);`
`SpinTennisGame.UI.cs:538 / HzPopup CourtPopup(string title, string message, Color accent, Sprite art, string icon, params (string label, Color color, Action action)[] buttons)`	`var t = HzUI.Label("Title", ribbon.transform, title, 66, Color.white); t.rectTransform.Stretch(6);`
`SpinTennisGame.UI.cs:544 / HzPopup CourtPopup(string title, string message, Color accent, Sprite art, string icon, params (string label, Color color, Action action)[] buttons)`	`p.Message = HzUI.Label("Msg", card.transform, message, 40, Ink, TextAnchor.MiddleCenter, false);`

`SpinTennisGame.UI.cs:587 / void UnlockPopup(int b, Action done)`	`CourtPopup("New: " + BoostName[b] + "!", BoostDesc[b] + "\nYou get " + BoostStart[b] + " free!", HzUI.Hex(BoostColor[b]), Art(BoostArt[b]), BoostIcon[b],`
`SpinTennisGame.UI.cs:599 / void VersusPopup(Action done)`	`var pop = CourtPopup("Stop " + Level + " - " + shortMode[(int)_def.Mode], msg, Teal, null, null, ("Play!", Coral, done));`
`SpinTennisGame.UI.cs:610 / void VersusPopup(Action done)`	`var ini = HzUI.Label("Init", av.transform, o.name.Substring(0, 1).ToUpperInvariant(), 96, Color.white); ini.rectTransform.Stretch();`
`SpinTennisGame.UI.cs:612 / void VersusPopup(Action done)`	`var styleL = HzUI.Label("Style", panel.transform, MatchDef.StyleName(o.Style), 28, OppColor, TextAnchor.MiddleCenter, false); styleL.fontStyle = FontStyle.Bold;`
`SpinTennisGame.UI.cs:614 / void VersusPopup(Action done)`	`var nm = HzUI.Label("Name", panel.transform, o.name, 48, Ink, TextAnchor.MiddleLeft);`
`SpinTennisGame.UI.cs:616 / void VersusPopup(Action done)`	`var vn = HzUI.Label("Venue", panel.transform, _venue.name + (Mathf.Abs(_def.wind) > 0.05f ? " - windy" : ""), 28, HzUI.Hex("#5A6A8A"), TextAnchor.MiddleLeft, false); vn.fontStyle = FontStyle.Bold;`
`SpinTennisGame.UI.cs:623 / void VersusPopup(Action done)`	`var lab = HzUI.Label("L", panel.transform, names[i], 24, Ink, TextAnchor.MiddleLeft, false); lab.fontStyle = FontStyle.Bold;`
`SpinTennisGame.UI.cs:647 / void ShowSettingsPopup()`	`var p = CourtPopup("Settings", "", Ink, null, null,`
`SpinTennisGame.UI.cs:648 / void ShowSettingsPopup()`	`(SoundLabel(), HzUI.Hex("#4FA8FF"), null),`
`SpinTennisGame.UI.cs:649 / void ShowSettingsPopup()`	`(MusicLabel(), HzUI.Hex("#9B6BFF"), null),`
`SpinTennisGame.UI.cs:660 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`SpinTennisGame.UI.cs:661 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/spintennis through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/spintennis/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/spintennis/Music/tour_loop.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.spintennis`, version, scene orientation and signing. The build command writes `Builds/Android/spintennis.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/spintennis/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.